# CampusEats

Design Document — Services, Contracts & Schema

CS 543 Web Services — Assignment 2

This document defines the capability list, service decomposition, service contracts, the full specification of the central placeOrder operation, and a validation of each service against the five properties of a well-formed service.

# 1. Capabilities

The following distinct jobs (capabilities) the CampusEats system performs, derived from the Assignment 1 brief:

- registerUser / authenticateUser — create accounts and verify identity for students, vendors, delivery partners and admins
- manageVendorProfile — create and update vendor details (name, type, location, opening hours, active status)
- manageMenu — add, update, and remove menu items for a vendor, including price and availability
- placeOrder — build a cart into a confirmed order, applying coupons and charging payment
- trackOrderStatus — report the current lifecycle status of an order to the student
- processPayment — charge a payment method (wallet, card, UPI) for an order and record the result
- applyCoupon — validate a coupon code and compute a discounted order total
- assignDelivery / trackDelivery — assign a delivery partner to an order and report delivery progress
- submitReview — let a student rate and comment on a completed order
- manageSubscription — create and track a recurring meal-plan subscription for a student at a vendor

## 2. Services & Team Members

CampusEats is decomposed into five services (within the 3–6 service limit). Each service owns its own data exclusively; no two services share a table.

| **Service** | Owns | Team Member |
|---|---|---|
| Identity Service | users | Team Member 1 |
| Catalogue Service | vendors, menu_items, reviews | Team Member 2 |
| Orders Service | carts, cart_items, orders, order_items | Team Member 3 |
| Payments Service | payments, wallets, wallet_transactions, coupons, subscriptions | Team Member 4 |
| Delivery Service | deliveries, delivery_zones, tracking_updates | Team Member 5 |

*Replace the placeholder "Team Member N" entries above with actual team member names before submitting.*

# 3. Contracts

Each service exposes the following operations to other services. Only the operation, its inputs, outputs and error cases are exposed — internal tables, ID formats, and SQL are hidden.

## Orders Service (main service — 4 operations)

| Operation | Input | Output | Errors |
|---|---|---|---|
| placeOrder(studentId, vendorId, items[{itemId, quantity}], deliveryType, timeSlot?, couponCode?, paymentMethod) | Student ID, vendor ID, ordered items, delivery type, optional time slot, optional coupon code, payment method | orderId, status, finalAmount, estimatedReadyTime | VENDOR_CLOSED, ITEM_UNAVAILABLE, INVALID_TIME_SLOT, INVALID_COUPON, PAYMENT_FAILED |
| getOrderStatus(orderId) | Order ID | status, lastUpdated | ORDER_NOT_FOUND |
| updateOrderStatus(orderId, newStatus) | Order ID, new status | success | ORDER_NOT_FOUND, INVALID_STATUS_TRANSITION |
| getOrderHistory(studentId) | Student ID | list of order summaries | USER_NOT_FOUND |

## Catalogue Service (main service — 4 operations)

| Operation | Input | Output | Errors |
|---|---|---|---|
| checkItemAvailability(vendorId, itemId) | Vendor ID, item ID | available, currentPrice | VENDOR_NOT_FOUND, ITEM_NOT_FOUND |
| getVendorMenu(vendorId) | Vendor ID | list of menu items | VENDOR_NOT_FOUND |
| updateItemAvailability(vendorId, itemId, isAvailable) | Vendor ID, item ID, availability flag | success | NOT_AUTHORIZED, ITEM_NOT_FOUND |
| submitReview(orderId, vendorRating, comment) | Order ID, rating, comment | reviewId | ORDER_NOT_COMPLETED, DUPLICATE_REVIEW |

## Identity Service (1 operation)

| Operation | Input | Output | Errors |
|---|---|---|---|
| authenticateUser(email, password) | Email, password | token, role, userId | INVALID_CREDENTIALS |

## Payments Service (2 operations)

| Operation | Input | Output | Errors |
|---|---|---|---|
| initiatePayment(orderId, amount, method) | Order ID, amount, payment method | paymentStatus, transactionRef | PAYMENT_FAILED, INSUFFICIENT_BALANCE |
| applyCoupon(code, orderTotal) | Coupon code, order total | discountedTotal | COUPON_INVALID, COUPON_EXPIRED |

## Delivery Service (1 operation)

| Operation | Input | Output | Errors |
|---|---|---|---|
| createDeliveryRequest(orderId, dropZone) | Order ID, drop zone | deliveryId, assignedPartnerId, eta | NO_PARTNER_AVAILABLE, INVALID_ZONE |

# 4. Full Specification: placeOrder

## Inputs

studentId, vendorId, items (a list of {menuItemId, quantity} pairs), deliveryType (PICKUP or DELIVERY), timeSlot (optional — required only for scheduled pickup/delivery), couponCode (optional), paymentMethod.

## Outputs

orderId, status = PLACED, finalAmount, estimatedReadyTime, and tokenNumber (issued only for pickup orders).

## Error Cases

- VENDOR_CLOSED — the vendor is outside its opening hours or marked inactive
- ITEM_UNAVAILABLE — one or more requested items are out of stock
- INVALID_TIME_SLOT — the requested time slot is not offered by the vendor
- INVALID_COUPON — the supplied coupon code does not exist, has expired, or does not apply to this order
- PAYMENT_FAILED — the payment could not be completed by the chosen method
- USER_NOT_FOUND — the student ID does not correspond to a known, authenticated user

## Internal Details Hidden From Callers

- Server-side re-validation of item prices — the operation never trusts a client-submitted price and always re-reads the current price from Catalogue
- The orchestration sequence: check availability → reserve items → apply coupon → charge payment → create delivery request or mark ready-for-pickup, with rollback if a later step fails
- The wallet-first-then-gateway fallback logic inside Payments (attempt wallet balance first, fall back to card/UPI gateway only if insufficient)
- The token-number generation algorithm used for pickup orders
- The delivery-partner assignment algorithm used when a delivery request is created

## 5. Service Validation Table

Each service is checked against the five properties of a service: reachable, self-contained, has a contract, independent, and loosely coupled. Failures are called out honestly, with a proposed fix.

| Service | Reachable | Self-contained | Has Contract | Independent | Loosely Coupled |
|---|---|---|---|---|---|
| **Identity Service** | ✔ | ✔ | ✔ | ✔ | PARTIAL — every other service calls it synchronously for user lookups, making it a coupling hotspot. Fix: validate JWT claims locally in each service instead of calling Identity on every request. |
| **Catalogue Service** | ✔ | ✔ | ✔ | ✔ | ✔ — only called by Orders for read-only availability checks, a natural dependency. |
| **Orders Service** | ✔ | ✔ | ✔ | PARTIAL — placeOrder synchronously chains Catalogue, Payments, Delivery and Identity in one call, so it cannot complete if any dependency is down. | PARTIAL — same root cause. Fix: use an event-driven/saga pattern (publish an OrderPlaced event; Payments and Delivery react asynchronously) instead of direct blocking calls. |
| **Payments Service** | ✔ | ✔ | ✔ | ✔ | ✔ — only receives inbound calls from Orders. |
| **Delivery Service** | ✔ | ✔ | ✔ | ✔ | PARTIAL — depends on Identity for delivery-partner profile lookups on every request. Same fix as Identity's issue (validate claims locally). |

**Summary:** Every service is reachable, self-contained, and has a defined contract. Catalogue and Payments are fully independent and loosely coupled. Identity, Orders, and Delivery each have a coupling weakness noted above, with a concrete fix proposed for each.